

SS-RESfM Code Map:

Changes to the RESfM Repository and Per-Model Code Paths

Project documentation

September 2026

1 Overview

Our repository (code/) is a fork of the authors’ released RESfM code (kept pristine at papers and code/resfm-main). This document records (§2) every functional change we made, and (§3) the exact code path, in execution order, behind each model we trained.

2 Changes to the RESfM repository

2.1 Self-supervised training (the core contribution)

- `loss_functions.py`: new `AdaptiveConfidenceWeightedOutliersLoss` — forms confident pseudo-labels from percentiles of the model’s own per-observation reprojection error (`loss.inlier_percentile/outlier_percentile`, default 20/80), with a scale-aware threshold-separation floor ($0.25 \times$ median error), a warm-up before pseudo-labeling starts, and optional contamination-linked percentiles. Plugged into the existing `CombinedLoss` ($\alpha \cdot \text{ESFMLoss} + \beta \cdot \text{classification}$; note: RESfM’s own supervised combination keeps the same class name — the supervised variant lives in `CombinedLossSupervised`). Selectable reprojection-weighting modes (`loss.reproj_weighting`).
- `train.py`: label-free validation — checkpoint selection by `our_repro` (mean reprojection error on validation scenes) with `train.validation_metric_higher_is_better=False`; RESfM’s original Accuracy selection retained for faithful runs (plus a fix: classification metrics are now actually computed at `epoch_evaluation` via `OutliersMetrics`, which the released code skipped).

2.2 Test-time outlier mechanisms (evaluated on any trained head)

- `train.py` (`epoch_evaluation`, TEST phase): after the network pass, we compute per-observation reprojection errors and write outlier verdicts to `Final_outliers.npz`. New conf-selected mechanisms: `test.outlier_source=mad` (pure MAD statistic, no head); `test.mad/std/huber_remove_head_weight` (robust-statistic removal + head reweighting of survivors — `madweight/stdweight/huberweight`); plain head removal / weighting / hybrid via existing thresholds (`test.removal_threshold`).
- `datasets/Euclidean.py` (`get_raw_data`): the FINE_TUNE stage consumes `Final_outliers.npz` — removal masks zero observations, weighting masks down-weight them in the fine-tune loss; hybrid combines both. GT-label removal (`remove_outliers_gt`) retained for ESfM* runs.

2.3 Evaluation pipeline and infrastructure

- `run_multiscene_eval.sh`: per-scene fleet launcher — instantiates a conf per (template, checkpoint dir, scene, seed), submits LSF jobs (`-jobprefix`, queue policy), heals missing scenes idempotently.
- Conf templates (`confs/*.conf.template`): per-dataset $\times$ mechanism $\times$ architecture evaluation configs (MegaDepth + five OOD datasets).

- Multi-seed protocol: per-seed training confs (random_seed), seed-suffixed eval roots; paired-seed evaluation supported via -seed (re-seeds the eval conf).
- Babysitter chain: resume_stalled_trainings.sh + trigger_ablation_resume.sh — resubmit preempted trainings from the latest checkpoint (resume=true), completion markers, congestion guard.
- Checkpoints now embed provenance (conf_snapshot, checkpoint_provenance) and the best-validation state.
- Lean outputs (default; eval.save_eval_diagnostics=True restores the full dumps): per-scene eval no longer writes forFigures camera dumps, plots, or epoch-checkpoint archives, and Final_outliers.npz stores masks + a valid_mask instead of copying M/Ns/Ps_gt (~ 1 GB $\rightarrow \sim 15$ MB per scene).
- Multi-GPU training via Lightning Fabric DDP (scenes sharded across ranks).

2.4 Data builders and datasets

- OOD track builders (1DSfM incl. the 5-scene hard split, Strecha, BlendedMVS, Olsson, ETH3D, VGG, Tanks&Temples): raw downloads $\rightarrow$ track matrices in RESfM’s npz format, with measured outlier_pct; multitrain/ merges pools for the diversity study.
- build_esfm_star_tracks.py: GT-cleaned tracks (ESfM* protocol).
- build_selfclean_tracks.py: tracks cleaned by a trained head’s predictions (two-stage protocol; -srcdir for multi-dataset pools).

2.5 Baselines, analysis, and verification

- colmap_baseline.py / glomap_baseline.py: classical SfM on our exact tracks (GT intrinsics), with reflection-aware Umeyama alignment and a gauge-free relative-rotation metric.
- Aggregators and studies: seed-band assembly, label-degradation study (aggregate_label_degradation.py), selector study (selector_study.py), per-scene table generators, and verify_iclr_tables.py (recomputes every paper table cell from saved evals).
- Architecture flags on models/SetOfSet.py: DeepSetOfSetOutliersNet exposes use_layer_norm/use_residual/dropout_rate; block/depth configurable for the 2×3 deep variants; original classes untouched for byte-faithful baselines.

3 Per-model code paths (in execution order)

All trainings share the spine

conf $\rightarrow$ multiple_scenes_learning.py $\rightarrow$ datasets/SceneData+Euclidean $\rightarrow$ models/SetOfSet.py $\rightarrow$ loss_functions.py $\rightarrow$ train.py loop,

and all evaluations share

run_multiscene_eval.sh $\rightarrow$ per-scene conf $\rightarrow$ train.py (TEST: head $\rightarrow$ Final_outliers.npz) $\rightarrow$ Euclidean.get_raw_data (mechanism)

The table lists what differs per model.

Model	Network class	Training loss	Ckpt selection	Distinguishing steps
SS-RESfM arms (one training; five eval mechanisms)	SetOfSetOutliersNet (1 $\times$ 3)	CombinedLoss + AdaptiveConfidence-WeightedOutliersLoss	our_repro	Train on MegaDepth-27, no labels. Eval per mechanism: madweight (test.mad_remove_head_weight), weight / weight+TTT (head scores as weights), remove / remove+TTT (head threshold 0.6), hybrid; TTT = per-scene fine-tune iterations before BA.

RESfM-scratch (declared baseline)	SetOfSetOutliersNet	CombinedLoss-Supervised (BCE on COLMAP labels)	our_repro	Identical pipeline; labels from outliers2 in the npz; eval = head removal per RESfM.
RESfM-faithful	same	same	Accuracy (authors' margin 10^{-3})	Only the validation metric/margin differ from scratch; requires the Outliers-Metrics fix.
RESfM @ 10^{-4} / 30k	same	same	our_repro	Base lr and/or 30k schedule (milestones rescaled).
ESFM* (+af)	SetOfSetNet (vanilla, no head)	ESFMLoss	our_repro	build_esfm_star_tracks.py first (GT-cleaned npz); af variant = authors' in-place zeroing construction.
Deep variants (deep-27, deep-multids, deep-ln)	DeepSetOfSetOutliersNet (2×3; LN/residual/dropout flags) or plain class with num_blocks=2	CombinedLoss-Supervised	our_repro	multids runs read datasets/multitrain; eval uses the deep conf templates and esfm_outliers_deep arch tag.
Multids (shallow)	SetOfSetOutliersNet	supervised or adaptive (per run)	our_repro	52-scene multitrain pool (MD27+ETH3D+VGG+T&T); af-multids replicates the authors' full config incl. Accuracy selection.
Two-stage family (selfclean)	stage 1 as faithful; stage 2 same class from scratch	stage 1 supervised; stage 2 supervised on cleaned tracks	stage 1 Accuracy; stage 2 our_repro	Order: train stage 1 → build_selfclean_tracks.py (stage-1 best head cleans the training npz, threshold 0.5) → train stage 2 on *_selfclean dataset → standard eval. Variants: 20k+20k and compute-matched 10k+10k (halved milestones); deep and multids versions swap the class/pool as above.
Classical rows	—	—	—	colmap_baseline.py/glomap_baseline.py build a COLMAP database from the same npz tracks → incremental / global mapping → reflection-aware alignment → JSON.

Reading order for a single evaluation (any learned model): (1) the fleet script writes a per-scene conf from the template, pointing train_results at the training's models/ best checkpoint; (2) multiple_scenes_learning.py -phase FINE_TUNE loads the scene npz; (3) a TEST pass of the checkpoint produces outlier verdicts (Final_outliers.npz); (4) get_raw_data applies the mechanism (remove/weight/hybrid) to the tracks; (5) per-scene fine-tuning (1,001 iters, reprojection loss only); (6) robust BA (pyceres/pycolmap); (7) evaluation.py writes Results_FINE_TUNE_stage_1_*.xlsx with pre-BA (ts_mean) and post-BA (ts_ba_final_mean) errors, rotation, and registered-camera counts — the source of every number in the papers.